Importance of Using Appropriate Cryptographic Solutions

4.1. Introduction

- In the modern digital world, **data is constantly transmitted, stored, and shared across networks**.
- This exposes sensitive information to threats such as **data breaches, hacking, identity theft, and unauthorized access**.
- **Cryptographic solutions** protect data by converting it into a secure form that only authorized users can access.
- These solutions ensure three main security goals:
    - **Confidentiality** – Data remains private.
    - **Integrity** – Data cannot be altered without detection.
    - **Authentication** – Identity of users or systems is verified.

Cryptography therefore acts as a **fundamental security mechanism for protecting digital communication and information systems**.

4.2. Role of Cryptography in Information Security

Cryptographic techniques protect digital information in several ways:

- **Encryption** protects data during storage and transmission.
- **Hashing** ensures that data has not been altered.
- **Digital signatures** verify the sender's identity.
- **Key management systems** securely manage encryption keys.
- **Obfuscation techniques** hide sensitive code or information.

Together, these mechanisms create a **secure environment for digital transactions and communications**.

4.3. Public Key Infrastructure (PKI)

Public Key Infrastructure (PKI) is a framework used to manage digital certificates and encryption keys.

PKI enables:

- Secure communication
- User authentication
- Data encryption
- Digital signatures

Organizations often establish their **own internal PKI** to gain better control over security policies and certificate management.

Advantages of Implementing PKI

- Enables **secure identification of users, devices, and servers**
- Allows organizations to **issue and manage digital certificates**
- Provides **strong encryption and authentication mechanisms**
- Enables quick **certificate revocation during security incidents**
- Ensures secure digital transactions and communication channels

PKI uses **asymmetric encryption**, which works with a **pair of keys**.

4.4. Key Escrow

Key escrow refers to a **trusted third party storing backup copies of encryption keys**.

Purpose of Key Escrow

It allows recovery of encrypted data if:

- The key owner forgets the password
- Hardware storing the key fails
- The key is lost
- The key owner becomes unavailable

How It Works

- Cryptographic keys are stored securely by a **trusted authority**.
- If necessary, the authority can **recover and restore access**.

Hardware Security Module (HSM)

Key escrow systems often use **Hardware Security Modules (HSMs)**.

An **HSM** is a specialized hardware device that:

- Protects cryptographic keys
- Performs secure encryption operations
- Provides tamper-resistant security

HSMs are widely used in:

- Banking systems
- Government security systems
- Cloud encryption services

4.5. Encryption

- **Encryption** is the process of converting **plaintext (readable data)** into **ciphertext (unreadable data)** to protect information.
- Only someone with the **correct decryption key** can convert ciphertext back into readable form.
- It protects sensitive data during **transmission and storage**.

Types of Encryption

Encryption is mainly classified into two types:

1. Symmetric Encryption

- Uses the **same secret key** for encryption and decryption.
- The sender and receiver must both know the same key.

2. Asymmetric Encryption

- Uses **two different keys**:
  - **Public key** for encryption
  - **Private key** for decryption

4.6. Symmetric Key Encryption

- Symmetric encryption uses **one shared secret key** for both encryption and decryption.
- The message before encryption is called **plaintext**.
- After encryption it becomes **ciphertext**.

Important Terms

- **Encryption (Enciphering):** Converting plaintext into ciphertext.
- **Decryption (Deciphering):** Converting ciphertext back to plaintext.
- **Cryptography:** Study of encryption techniques.
- **Cryptanalysis:** Breaking encrypted messages without knowing the key.
- **Cryptology:** Combination of cryptography and cryptanalysis.

4.7. Symmetric Cipher Model

The symmetric encryption model consists of **five main components**:

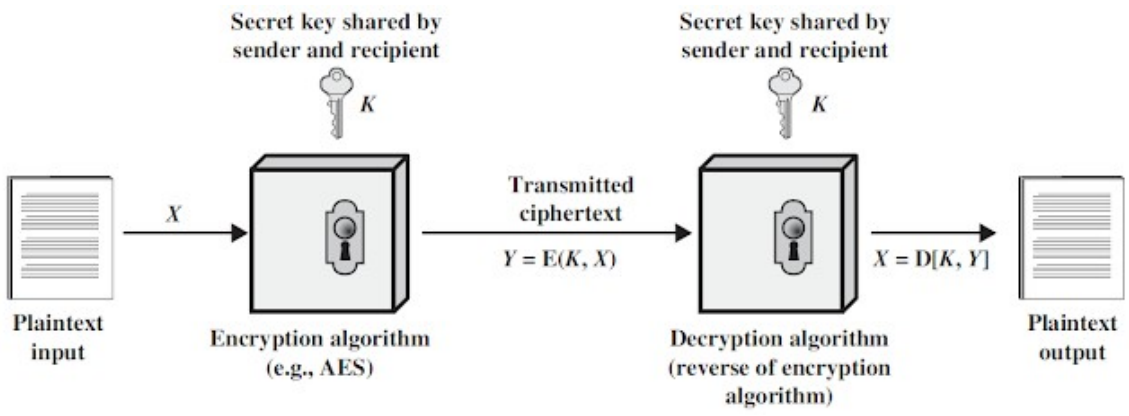


Figure 1: Symmetric Key Cryptography

1. Plaintext

- The **original readable message** that needs protection.

2. Encryption Algorithm

- Mathematical method used to **convert plaintext into ciphertext**.

3. Secret Key

- A **confidential key shared** between sender and receiver.

4. Ciphertext

- The **encrypted output** which appears random and unreadable.

5. Decryption Algorithm

- Mathematical process used to **convert ciphertext back into plaintext**.

4.8. Classical Encryption Techniques

Early encryption techniques fall into two main categories:

1. Substitution Ciphers

Letters are **replaced with other letters or symbols**.

Examples:

- Caesar Cipher
- Monoalphabetic Cipher
- Playfair Cipher
- Hill Cipher
- Polyalphabetic Cipher
- Vigenère Cipher
- Vernam Cipher

2. Transposition Ciphers

Letters are **rearranged in a different order without changing them**.

Examples:

- Rail Fence Cipher
- Row Column Cipher

Caesar Cipher

- The **Caesar Cipher** is one of the **oldest substitution ciphers**.
- Named after **Julius Caesar**, who used it in military communication.
- Each letter in plaintext is **shifted a fixed number of positions** in the alphabet.

Example:
Shift = 3

Plaintext
ABC → DEF

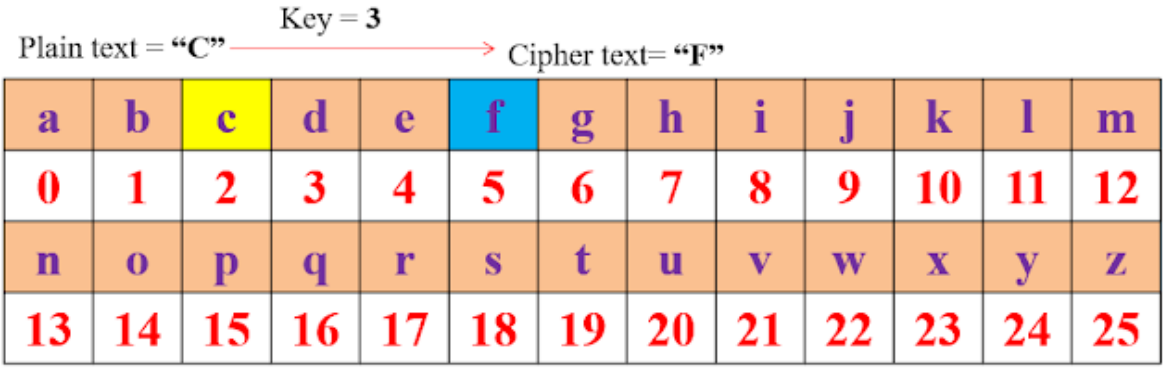


Figure 2: Caesar cipher

Mathematical Formula

Encryption

$C=(P+K) \bmod 26$

Decryption

$P=(C-K) \bmod 26$

Where:

- **P** = Plaintext letter
- **C** = Ciphertext letter
- **K** = Key (shift value)
- **mod 26** ensures wrapping within the alphabet.

Advantages of Caesar Cipher

- Very simple to understand.
- Easy to implement.
- Useful for learning basic encryption concepts.

Disadvantages

- Only **25 possible keys**, so brute force is easy.
- Vulnerable to **frequency analysis**.
- Not secure for modern applications.

Monoalphabetic Cipher

A **monoalphabetic cipher** is a substitution cipher where:

- Each plaintext letter is replaced with **another fixed letter**.
- The mapping remains **consistent throughout the message**.

Example mapping (Random):

Plaintext Alphabet
ABCDEFGHIJKLMNOPQRSTUVWXYZ

Cipher Alphabet
QWERTYUIOPASDFGHJKLZXCVBNM

Example:

Plaintext: HELLO
Ciphertext: ITSSG

Example mapping (Keyword):

Keyword : MONARCHY

Plaintext Alphabet
ABCDEFGHIJKLMNOPQRSTUVWXYZ

Cipher Alphabet
MONARCHYBDEFGIJKLPQSTUVWXZ

Example:

Plaintext: HELLO
Ciphertext: YRFFJ

Key Features

- Each plaintext letter corresponds to **one unique ciphertext letter**.
- Mapping stays the **same for the entire message**.
- More secure than Caesar cipher but still breakable.

Encryption Process

1. Define plaintext.
2. Create a **cipher alphabet** using a random key or keyword.
3. Replace each plaintext letter using the cipher alphabet.
4. Generate ciphertext.

Decryption Process

1. Take the ciphertext.
2. Use the **same cipher alphabet**.
3. Replace each ciphertext letter with its plaintext equivalent.
4. Recover the original message.

Advantages

- Easy to implement.
- Fast encryption and decryption.
- Much larger keyspace (**26! possible keys**).

Disadvantages

- Vulnerable to **frequency analysis**.
- Susceptible to **known plaintext attacks**.
- Repeated patterns reveal clues to attackers.

Playfair Cipher

The **Playfair Cipher** is a **digraph substitution cipher**.

- Encrypts **pairs of letters** instead of single letters.
- Developed by **Charles Wheatstone in 1854**.
- Uses a **5 × 5 matrix** built from a keyword.
- Usually **I and J are combined**.

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

Steps in Playfair Cipher

Step 1: Create Key Matrix

Example keyword: **MONARCHY**

Fill the matrix with keyword letters first, then remaining alphabet letters.

Step 2: Prepare Plaintext

Rules:

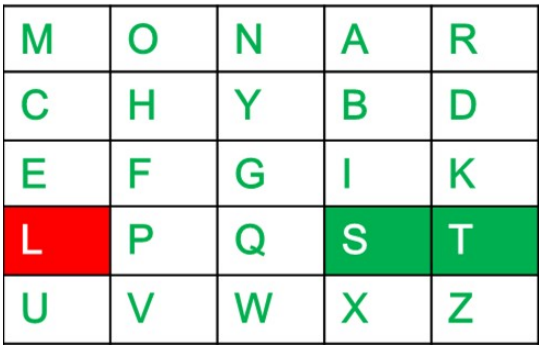
1. Divide message into **letter pairs (digraphs)**.
2. If both letters in a pair are the same → insert **X**.
3. If message length is odd → add **X** at the end.

Example:

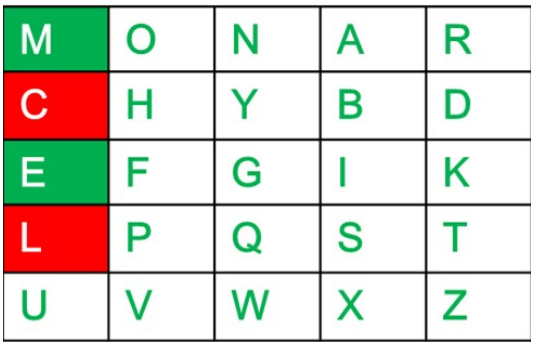
Plaintext: HELLO
Digraphs: HE, LX, LO

Step 3: Encryption Rules

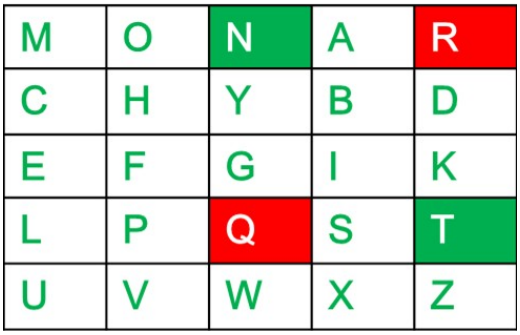
1. **Same Row**
   Replace letters with the letter to their **right**.



2. **Same Column**
   Replace letters with the letter **below**.



3. **Rectangle Rule**
   Replace with letters at **horizontal opposite corners of the rectangle**.



Example result:

Plaintext: HELLO
Ciphertext: CFUZMP

Advantages

- More secure than simple substitution ciphers.
- Encrypts **letter pairs**, reducing frequency analysis effectiveness.
- Can be done manually.

Disadvantages

- Still vulnerable to **digraph frequency analysis**.
- Manual process may cause errors.
- One letter (usually J) must be removed or merged.
- Considered **obsolete for modern cryptography**.

Vigenère Cipher

- The **Vigenère Cipher** is a classical cryptographic technique used to encrypt alphabetic text using a **polyalphabetic substitution method**.
- It was developed by **Blaise de Vigenère in the 16th century** and became one of the most widely used manual encryption techniques in historical cryptography.
- Vigenère cipher uses a **keyword to determine different shift values for different letters of the plaintext**.
- Because of this varying shift, it provides **stronger security and reduces the effectiveness of simple frequency analysis**.

1. Keyword Selection

A **keyword** is selected for encryption (for example, **KEY**).
The keyword is **repeated or truncated** so that its length matches the length of the plaintext.

Example:

Plaintext: HELLO
Keyword: KEY

2. Encryption Process

Each letter of the plaintext is shifted according to the **alphabetical position of the corresponding keyword letter**.

Alphabet numbering:

A = 0, B = 1, C = 2, … , Z = 25

The encryption formula is:

$C = (P + K) \bmod 26$

Where:

- **C** = Ciphertext letter
- **P** = Plaintext letter
- **K** = Key letter value
- **mod 26** ensures the result remains within the alphabet range.

3. Decryption Process

Decryption reverses the encryption process by subtracting the key value from the ciphertext value.

The decryption formula is:

P= (C- K) mod 26

Plaintext	**H**	**E**	**L**	**L**	**O**
Keyword	K	E	Y	K	E
Shift Value	10	4	24	10	4
Ciphertext	R	I	J	V	S

Advantages

- More secure than **monoalphabetic substitution ciphers**.
- Uses **multiple cipher alphabets**, which reduces predictable patterns.
- Makes **simple frequency analysis difficult** for short messages.
- Simple to implement manually.

Disadvantages

- If the **keyword is short**, patterns may repeat.
- Reusing the same keyword for multiple messages can compromise security.
- Not secure enough for modern cryptographic applications.

Transposition Cipher

In **Transposition Ciphers**, the letters are **not replaced but rearranged** to produce the ciphertext.

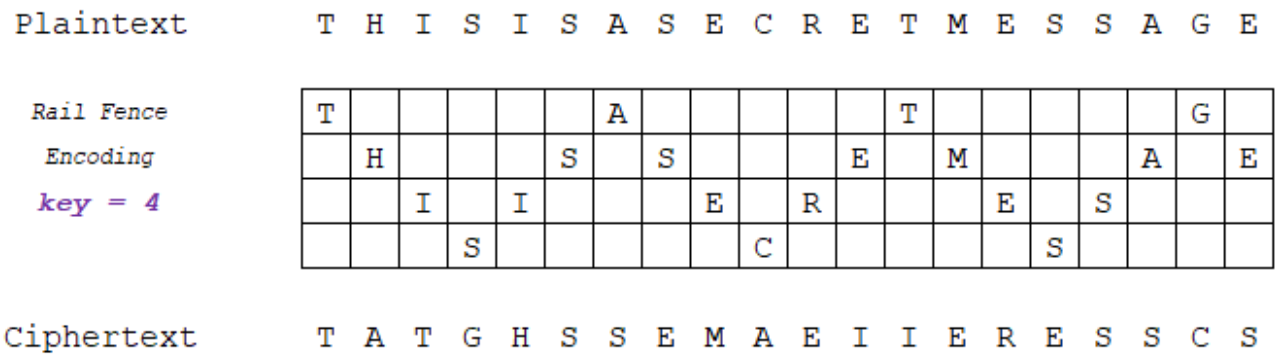
Rail Fence Cipher

The **Rail Fence Cipher** writes the plaintext in a **zigzag pattern across multiple rows (rails)**.

Steps:

1. Choose the number of rails (depth).
2. Write the message in a zigzag pattern.
3. Read the rows sequentially to produce ciphertext.

Example:



Row Column Transposition Cipher

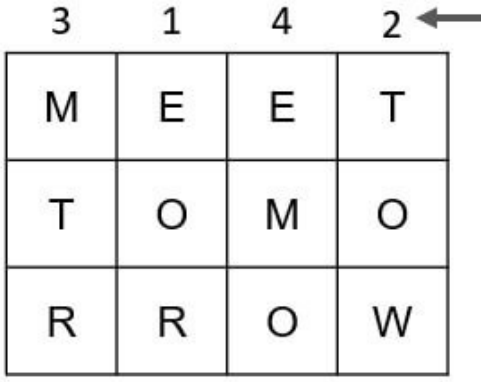
In this cipher:

1. The message is written **row by row in a table**.
2. The columns are **read in a specific order determined by a key**.

Example:

Plain text: MEET TOMORROW

Key: 3142



Cipher Text: EORTOWMTREMO

Stream Cipher

A **Stream Cipher** encrypts data **one bit or one byte at a time**. It converts plaintext into ciphertext by combining it with a **keystream** (a sequence of random or pseudo-random bits).

Characteristics

- Encrypts data **bit-by-bit or byte-by-byte**
- Requires a **continuous keystream**
- Very **fast and efficient**
- Suitable for **real-time communication**

Block Cipher

A **Block Cipher** encrypts data in **fixed-size blocks** (e.g., 64-bit or 128-bit blocks). Each block of plaintext is transformed into ciphertext using a secret key.

Characteristics

- Processes data in **blocks**
- Requires **padding** if data size is not equal to block size
- More **structured and secure** than stream ciphers

4.9. Feistel Cipher

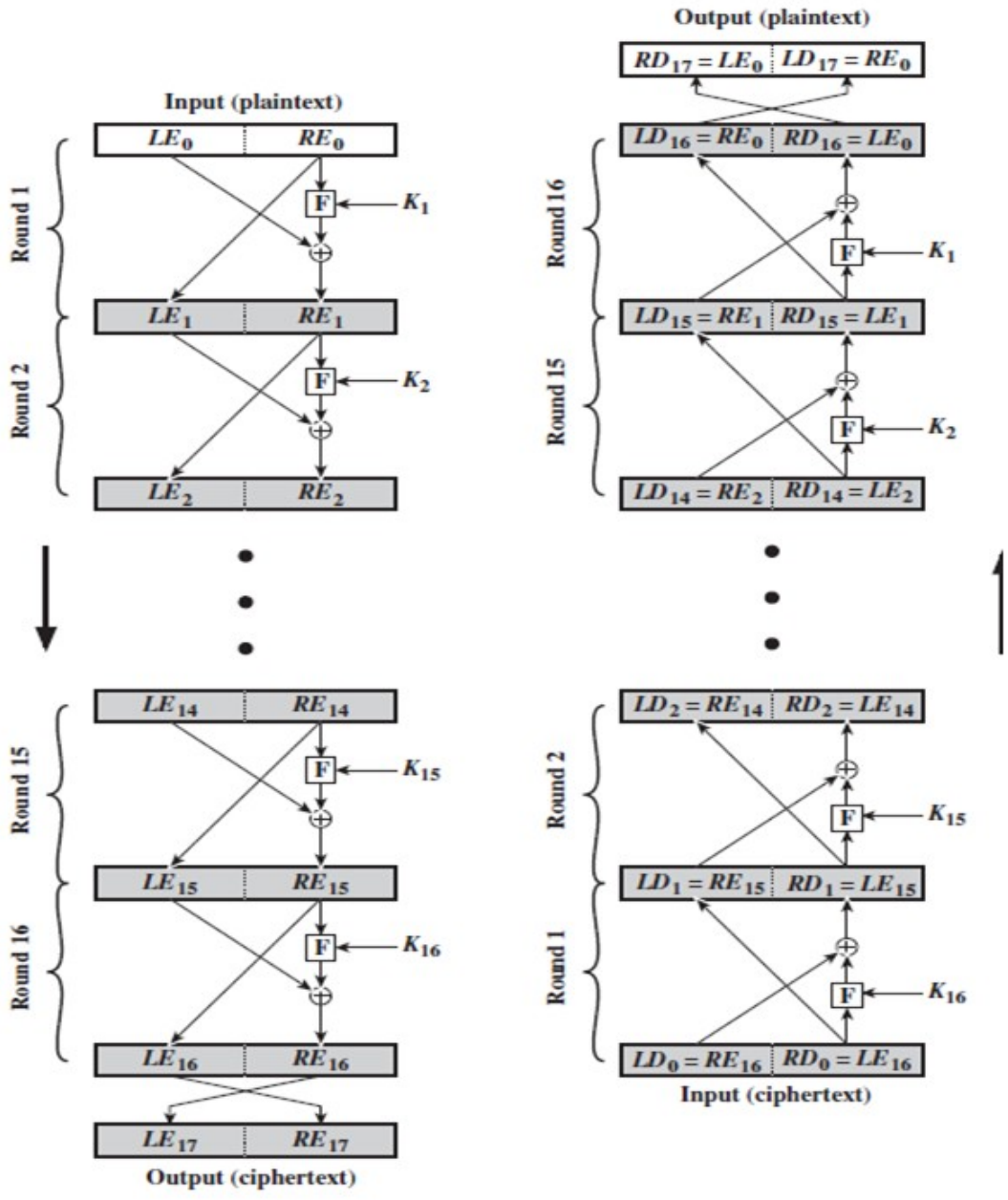
The **Feistel Cipher** is a symmetric encryption structure used in many modern block ciphers. It was developed by **Horst Feistel** and forms the basis of well-known algorithms like **Data Encryption Standard**.

A key advantage of the Feistel structure is that **the same algorithm can be used for both encryption and decryption**, making implementation efficient and practical.

Working Principle

For each round **i**:

1. The input block is divided:
    - Left = Li−1
    - Right = Ri−1
2. A round function **F** is applied to the right half along with a subkey.
3. The output is XORed with the left half.
4. The halves are swapped.



Diffusion and Confusion

The concepts of **Diffusion** and **Confusion** were introduced by **Claude Shannon** as fundamental principles for designing secure encryption systems.

These properties are achieved using **product ciphers**, which combine multiple encryption operations to increase security.

Diffusion

Diffusion is the property that aims to **hide the relationship between the plaintext and the ciphertext**.

Definition

Diffusion ensures that:

- A **small change in plaintext** results in a **large change in ciphertext**
- Each symbol in the ciphertext depends on **multiple symbols of the plaintext**

Confusion

Confusion is the property that aims to **hide the relationship between the ciphertext and the encryption key**.

Definition

Confusion ensures that:

- A **small change in key** results in a **large change in ciphertext**
- The relationship between key and ciphertext is highly complex

Achieving Diffusion and Confusion

Both properties are achieved using **product ciphers**, which combine multiple operations such as:

- **Substitution (S-boxes)** → provides **confusion**
- **Permutation (P-boxes)** → provides **diffusion**

Data Encryption Standard (DES)

The **Data Encryption Standard (DES)** is a **symmetric key block cipher** that was widely used for securing data. It was developed in the 1970s and later adopted as a federal standard by **National Institute of Standards and Technology** (originally by IBM with input from the NSA).

DES is based on the **Feistel cipher** structure and uses multiple rounds of processing to provide security.

Key Features of DES

- **Block size**: 64 bits
- **Key size**: 56 bits (actual key is 64 bits, but 8 bits are used for parity)

- **Number of rounds**: 16
- **Type**: Symmetric block cipher
- **Structure**: Feistel network

Working Principle of DES

DES encrypts data using a **series of permutations and substitutions** across multiple rounds.

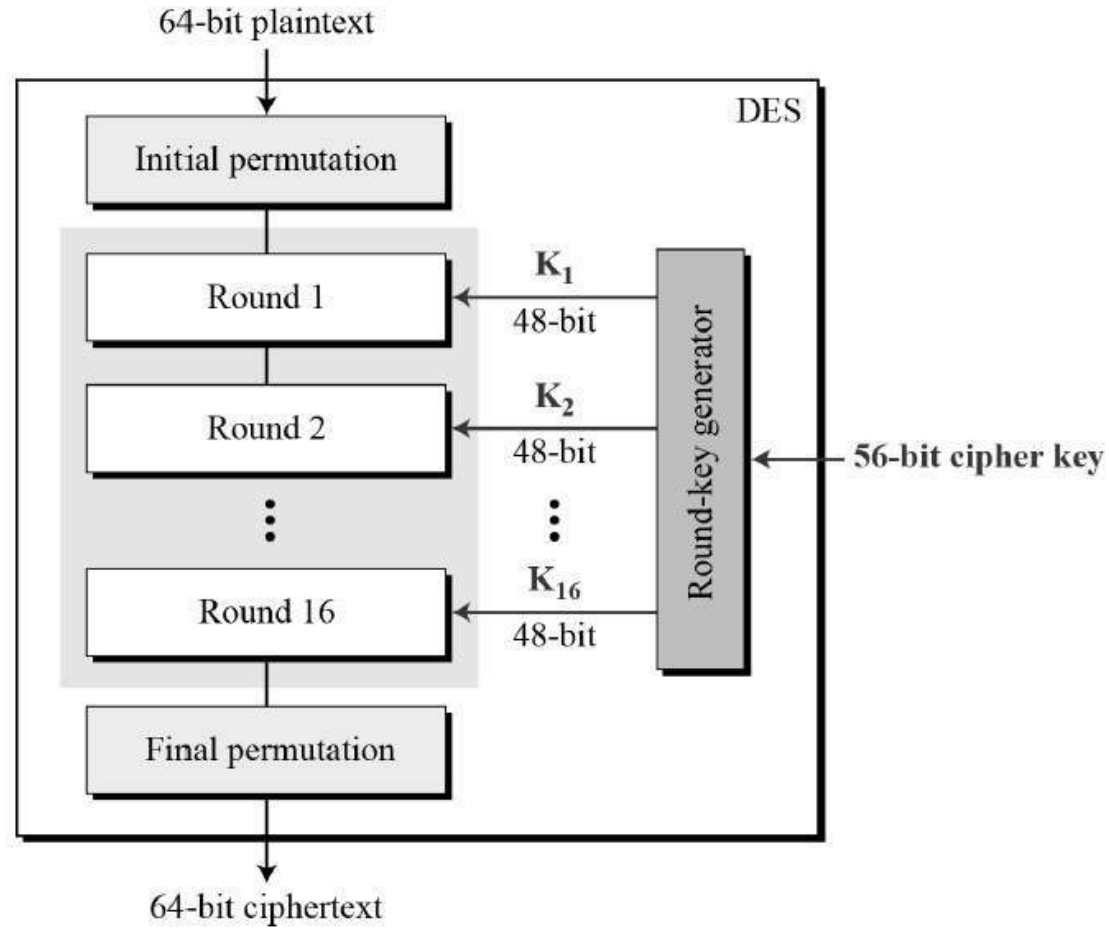


Figure 4: DES Structure

1. Initial Permutation (IP)

- The 64-bit plaintext undergoes an **initial permutation**
- Rearranges bits according to a predefined table

Initial Permutation

58	50	42	34	26	18	10	02
60	52	44	36	28	20	12	04
62	54	46	38	30	22	14	06
64	56	48	40	32	24	16	08
57	49	41	33	25	17	09	01
59	51	43	35	27	19	11	03
61	53	45	37	29	21	13	05
63	55	47	39	31	23	15	07

Figure 5: Initial Permutation Table

2. Splitting

- The permuted block is divided into:
    - Left half ($\mathbf{L_0}$)
    - Right half ($\mathbf{R_0}$)

3. 16 Feistel Rounds

Each round performs the following operations:

- Apply function **F** on the right half
- XOR the result with the left half
- Swap the halves

Round Equations

Where:

- $\mathbf{K_i}$ = Round key
- **F** = Round function

4. Function F (Mangler Function)

The function **F** includes:

1. **Expansion (P-box)**
    - Expands 32-bit input to 48 bits
2. **Key Mixing**
    - XOR with round key
3. **Substitution (S-boxes)**
    - Converts 48 bits back to 32 bits
    - Provides **confusion**
4. **Straight Permutation (P-box)**
    - Rearranges bits
    - Provides **diffusion**

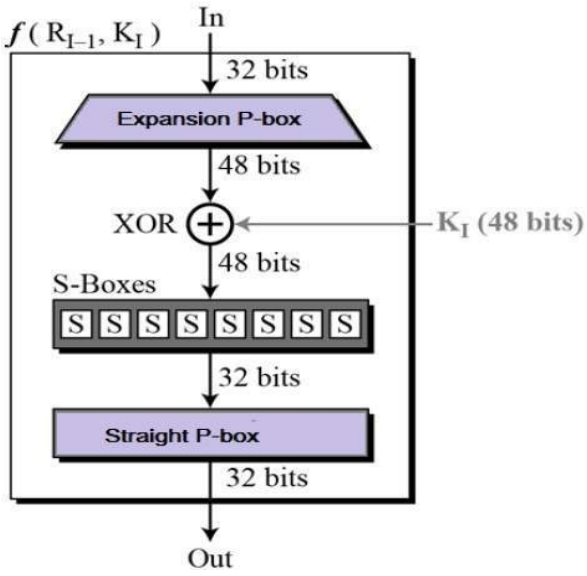


Figure 6: One Round of mangler function

5. Final Permutation (IP^{-1})

- After 16 rounds, the halves are combined
- A **final permutation** is applied to produce ciphertext

Final Permutation

40	08	48	16	56	24	64	32
39	07	47	15	55	23	63	31
38	06	46	14	54	22	62	30
37	05	45	13	53	21	61	29
36	04	44	12	52	20	60	28
35	03	43	11	51	19	59	27
34	02	42	10	50	18	58	26
33	01	41	09	49	17	57	25

Figure 7: Final Permutation

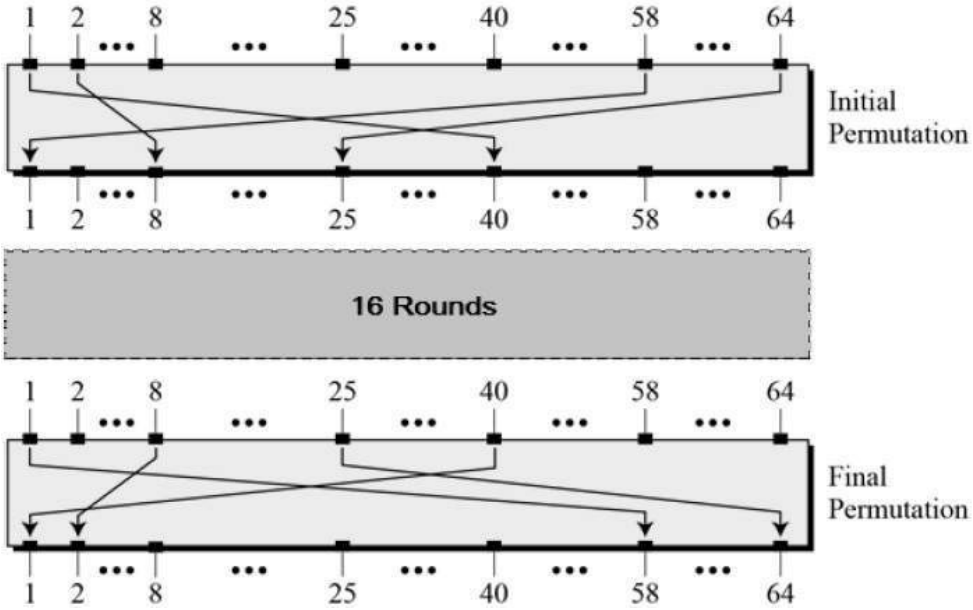


Figure 8: Initial and Final Permutation

Expansion (P-box) : Since right input is 32-bit and round key is a 48-bit, we first need to expand right input to 48 bits. Permutation logic is graphically depicted in the following illustration:

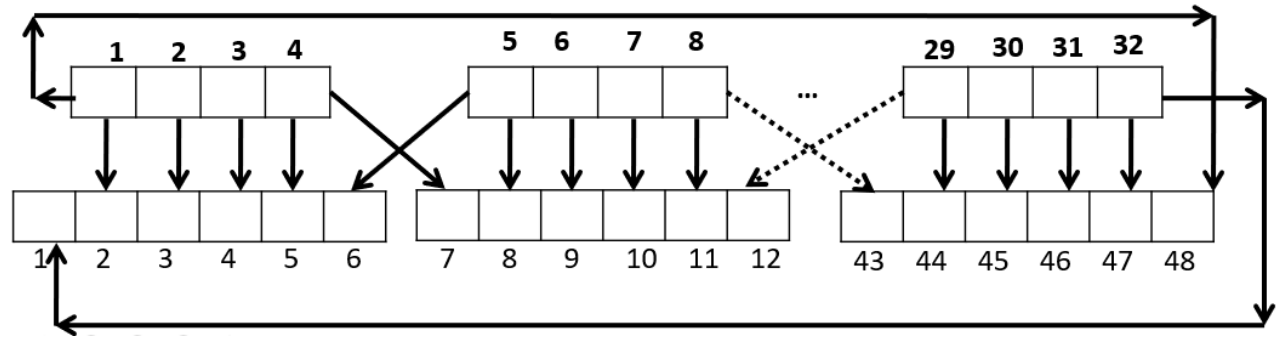


Figure 9: Expansion Permutation

32	01	02	03	04	05
04	05	06	07	08	09
08	09	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	31	31	32	01

Figure 10: Expansion P-box table

Substitution Boxes − The S-boxes carry out the real mixing (confusion). DES uses 8 S-boxes, each with a 6-bit input and a 4-bit output. Refer the following illustration.

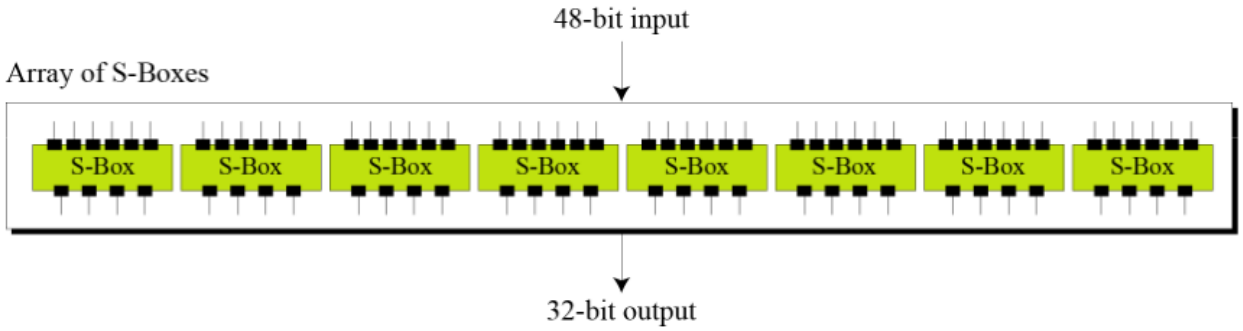


Figure 11: S-box

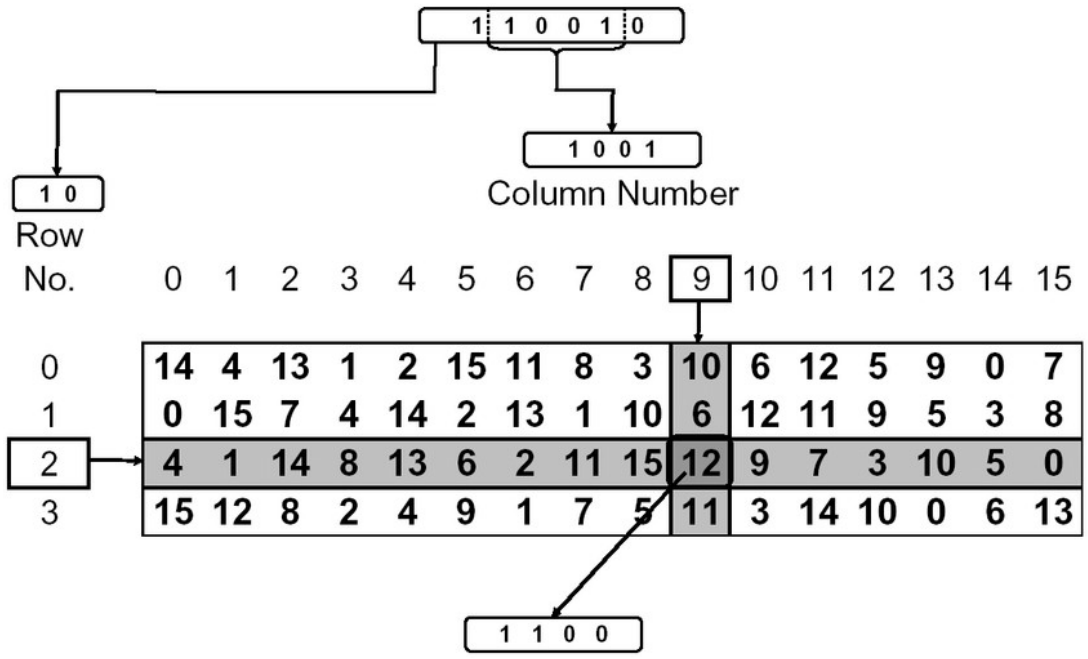


Figure 12: Substitution Box

Straight Permutation − The 32 bit output of S-boxes is then subjected to the straight permutation with rule shown in the following illustration:

16	07	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25

Figure 13: Straight permutation table

DES Key Generation

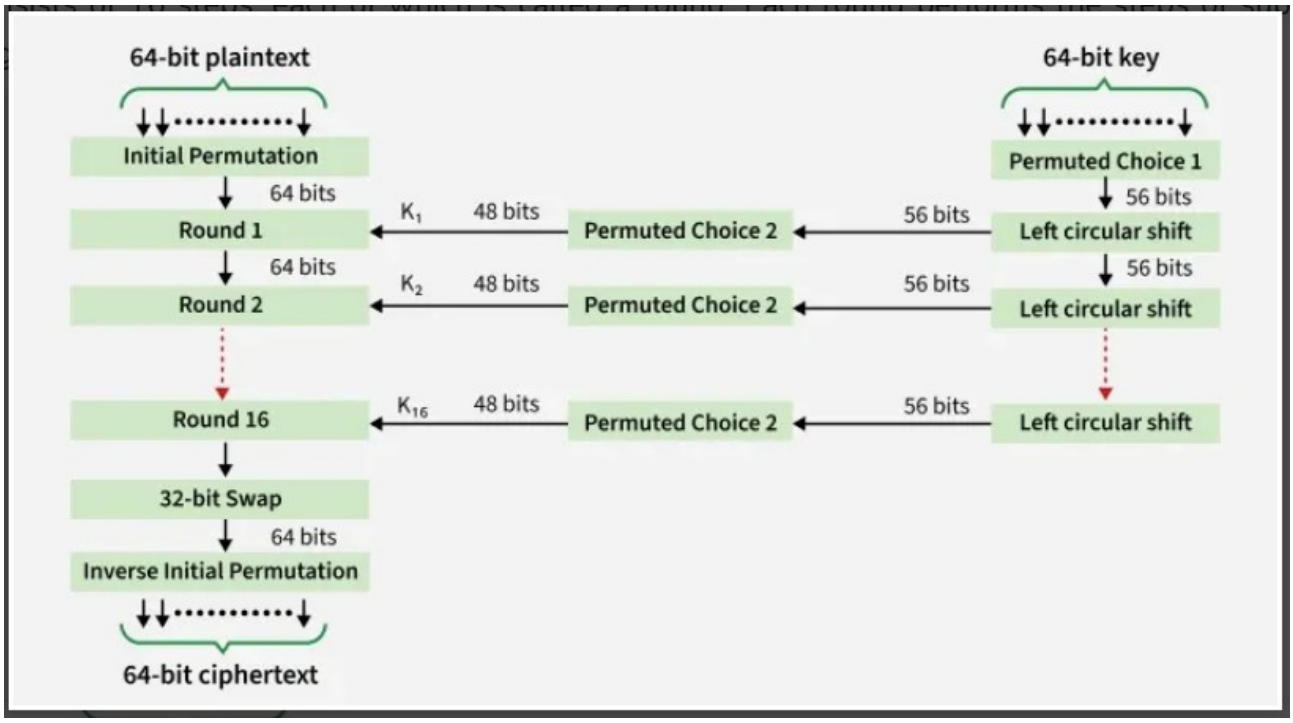


Figure 14: DES Encryption Process

1. **Initial 64-bit Key Input:** The process starts with a 64-bit key provided by the user. Every eighth bit (bits 8, 16, 24, 32, 40, 48, 56, and 64) is a parity bit and is discarded during the initial processing.
2. **Permuted Choice 1 (PC-1):** The 64-bit key is put through the PC-1 permutation, which rearranges the remaining bits and reduces the key size to 56 bits (by effectively ignoring the parity bits). This 56-bit result is the master key for the subsequent steps.
3. **Key Splitting:** The 56-bit key is divided into two 28-bit halves, labeled as C_0 (left half) and D_0 (right half).
4. **Circular Left Shifts:** For each of the 16 rounds of the DES algorithm, both C and D halves undergo a circular left shift (rotation). The number of shifts varies depending on the round number:
    - **One-bit left shift** for rounds 1, 2, 9, and 16.

- **Two-bit left shift** for all other rounds (3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15).

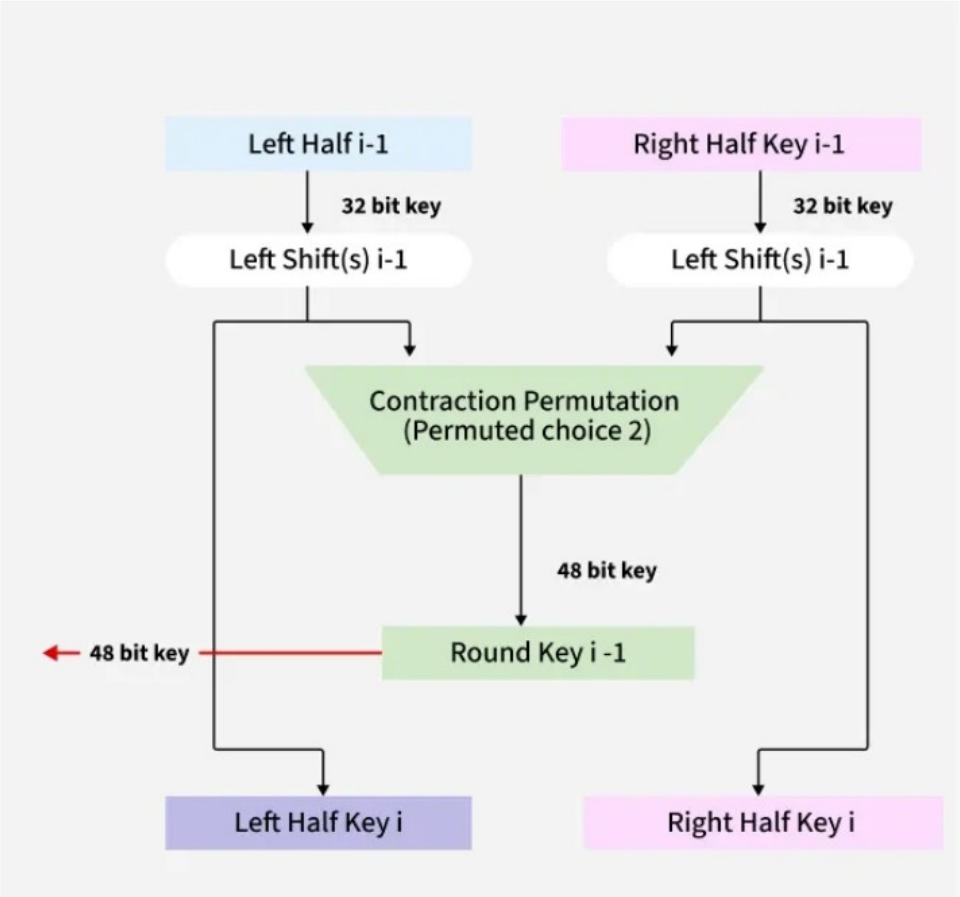


Figure 15: Key Transformation in DES

5. **Permuted Choice 2 (PC-2) / Compression:** After each shift operation, the two 28-bit halves (now C_i and D_i for round *i*) are combined into a 56-bit block and then passed through the PC-2 permutation table. The PC-2 table selects 48 bits from the 56-bit input, effectively dropping 8 bits. The 48-bit output from PC-2 is the specific subkey (K_i) used for that encryption round.

AES (Advanced Encryption Standard)

The **Advanced Encryption Standard (AES)** is a **symmetric key encryption algorithm** used to securely encrypt and decrypt data. It was adopted by the National Institute of Standards and Technology (NIST) in **2001** as a replacement for the older **Data Encryption Standard (DES)**.

AES is based on the **Rijndael algorithm**, developed by **Joan Daemen** and **Vincent Rijmen**.

It is widely used in:

- Banking systems
- Wireless security (Wi-Fi – WPA2/WPA3)
- Secure communication (SSL/TLS)
- Government and military applications

Key Features of AES

- **Symmetric key algorithm** (same key for encryption and decryption)
- Fixed **block size = 128 bits**
- Key lengths: **128, 192, 256 bits**
- Based on **Substitution–Permutation Network (SPN)**
- Highly secure and efficient
- Resistant to known cryptographic attacks

AES Structure

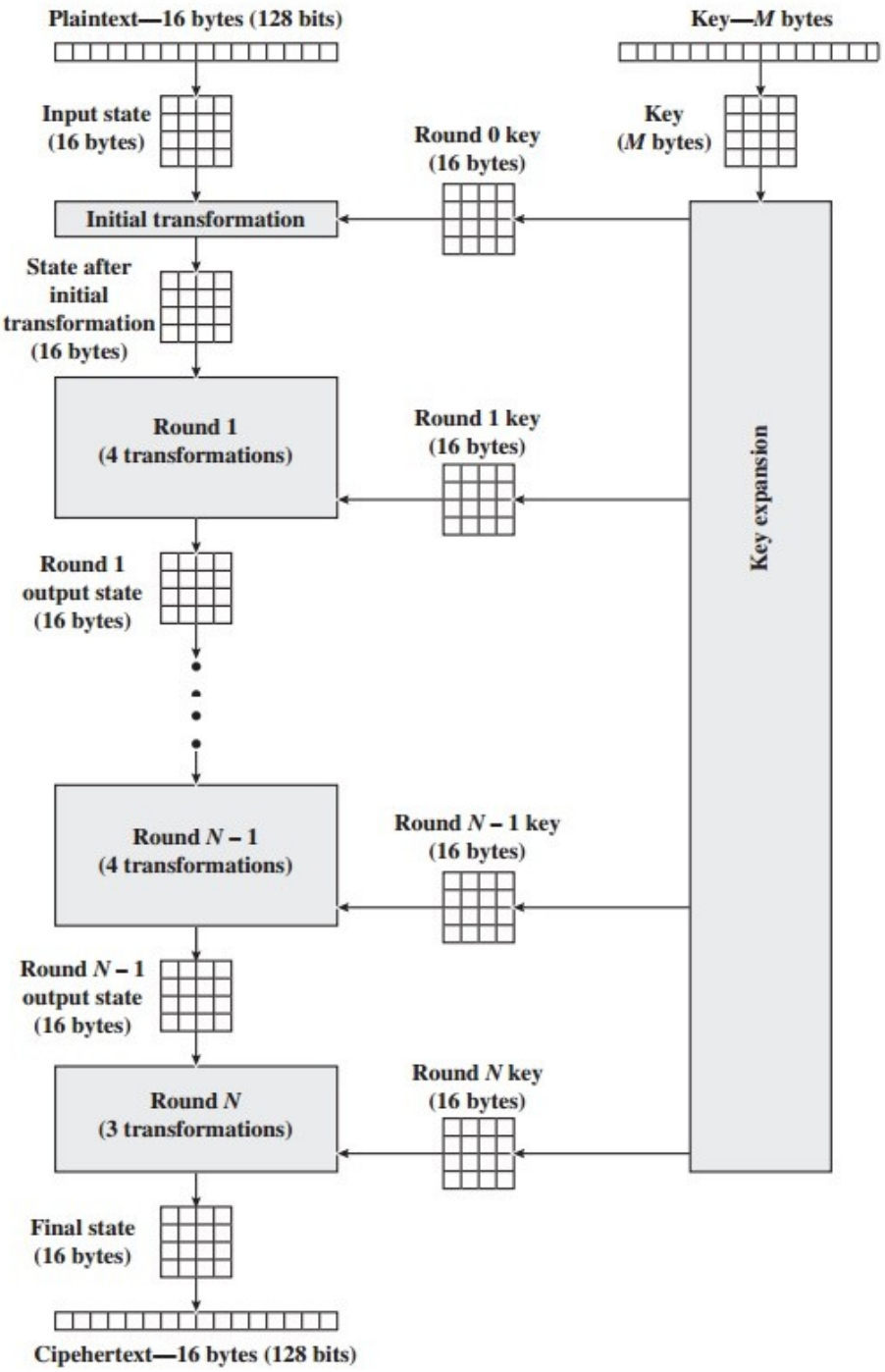


Figure 16: AES Overall Structure

AES performs operations on bytes of data rather than in bits. Since the block size is 128 bits, the cipher processes 128 bits (or 16 bytes) of the input data at a time.

The number of rounds depends on the key length as follows :

N (Number of Rounds)	Key Size (in bits)
10	128
12	192
14	256

Encryption

AES considers each block as a 16-byte (4 byte x 4 byte = 128) grid in a column-major arrangement.

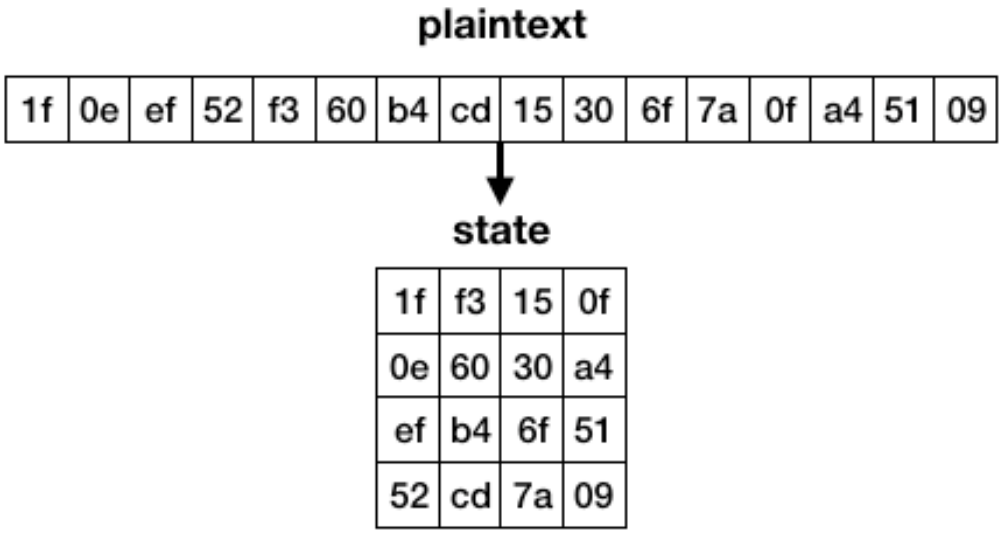


Figure 17: AES 4X4 Grid

Initial Round

- **AddRoundKey**
  - The plaintext matrix is XORed with the first round key:

This is the **only step where key is directly applied initially**.

Main Rounds (Repeated Steps)

Each round includes four transformations:

- Substitution Bytes (Sub)
- Shift Rows
- Mix Columns
- Add Round Key

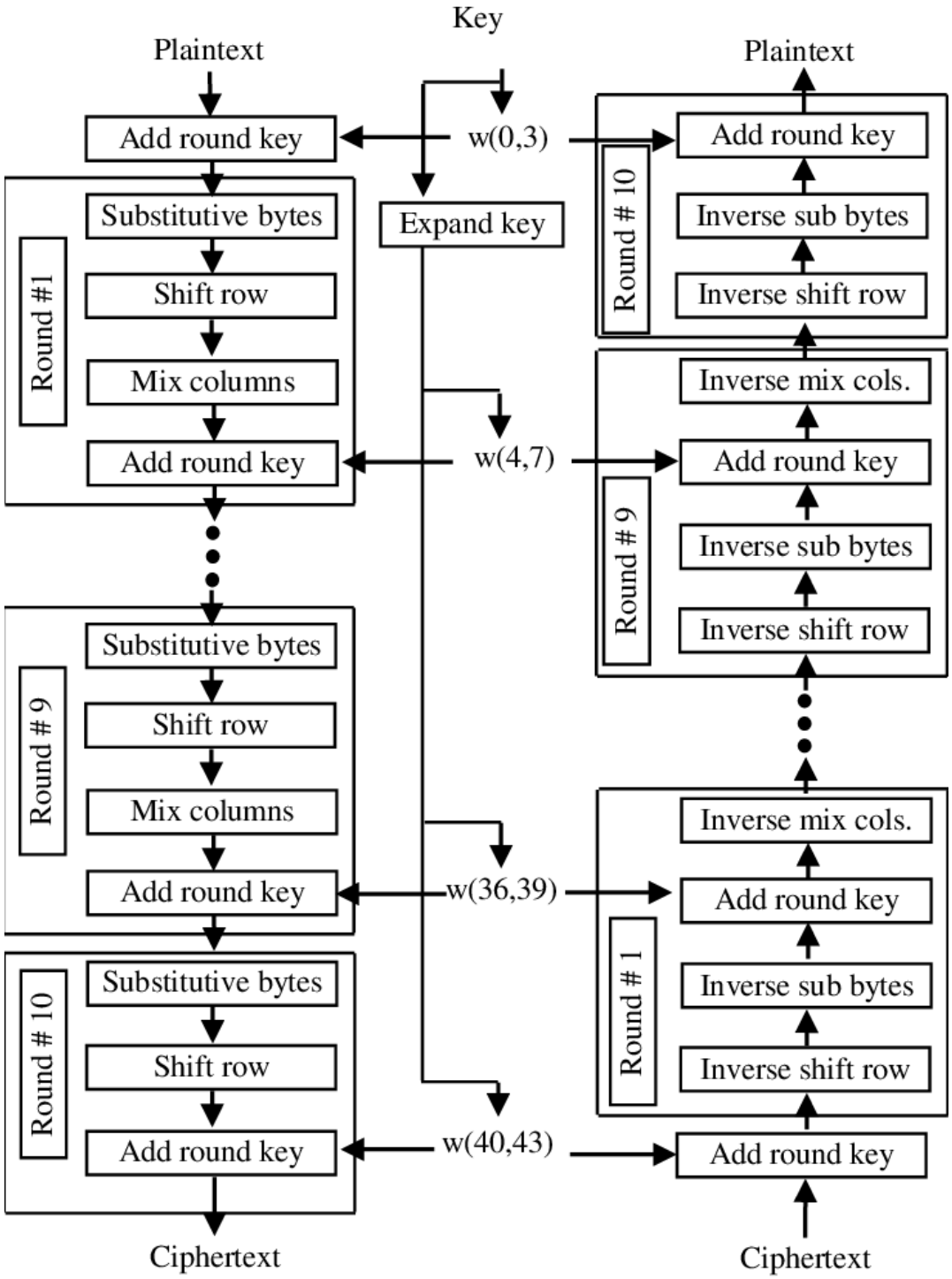


Figure 18: The AES Encryption and Decryption Process

Sub Bytes

This step implements the substitution. In this step, each byte is substituted by another byte. It is performed using a lookup table also called the S-box.

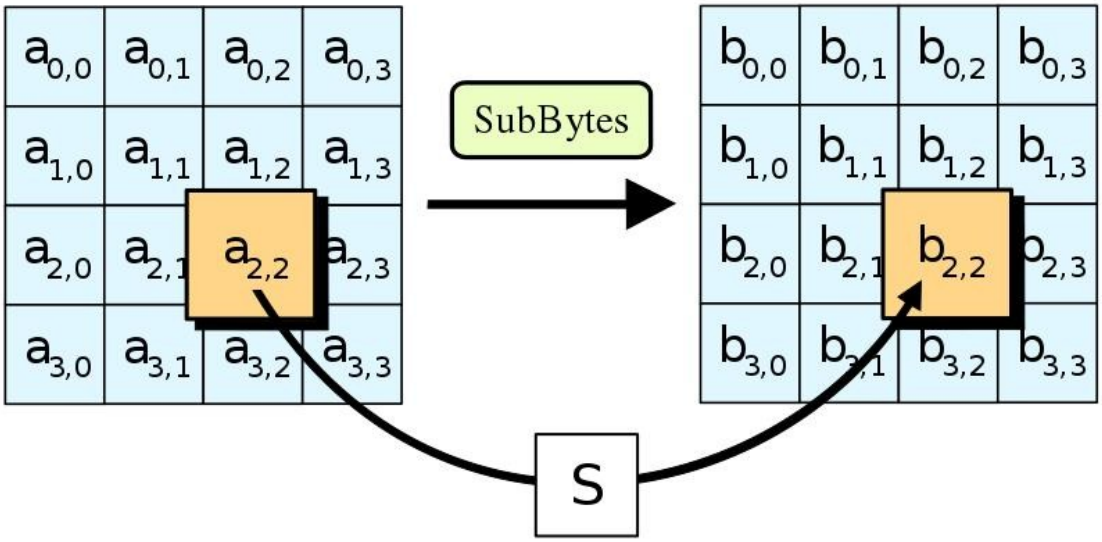


Figure 19: Sub Byte

Shift Rows

This step is just as it sounds. Each row is shifted a particular number of times.

- The first row is not shifted
- The second row is shifted once to the left.
- The third row is shifted twice to the left.
- The fourth row is shifted thrice to the left.

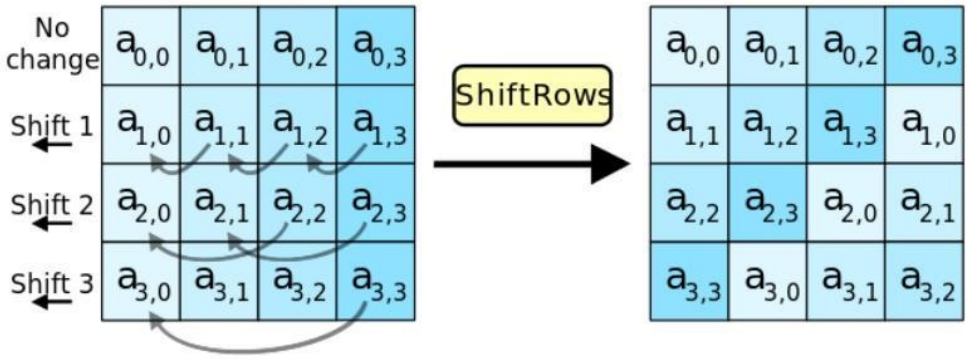


Figure 20: Shift Rows

Mix Columns

This step is a matrix multiplication. Each column is multiplied with a specific matrix and thus the position of each byte in the column is changed as a result.

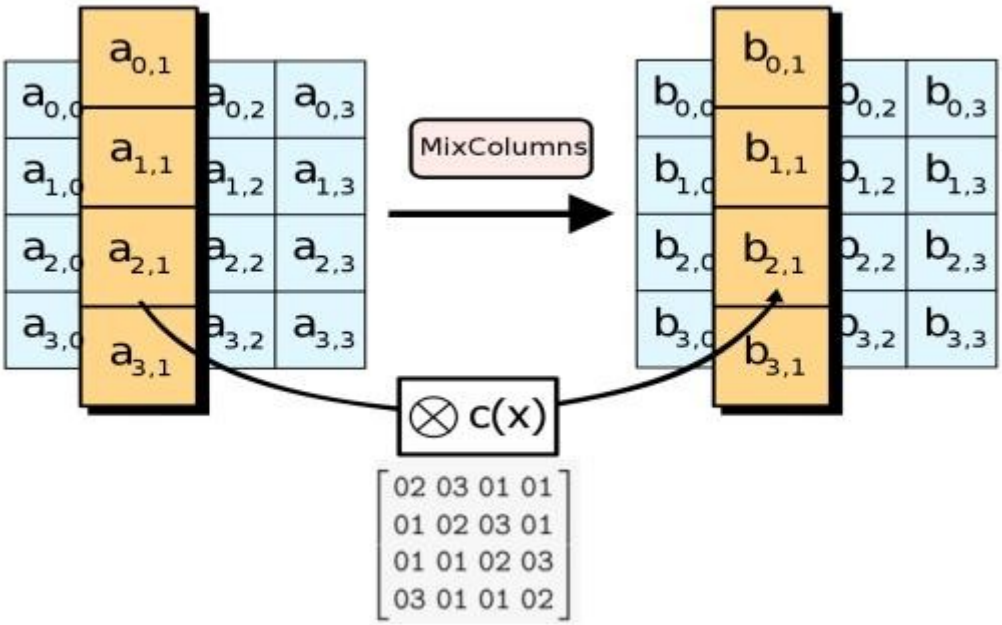


Figure 21: Mix Column

Add Round Keys

- Now the resultant output of the previous stage is XOR-ed with the corresponding round key. Here, the 16 bytes are not considered as a grid but just as 128 bits of data.
- After all these rounds 128 bits of encrypted data are given back as output. This process is repeated until all the data to be encrypted undergoes this process.

Final Round

- SubBytes
- ShiftRows
- AddRoundKey

(Note : MixColumns is NOT performed)

Decryption

The stages in the rounds can be easily undone as these stages have an opposite to it which when performed reverts the changes. Each 128 blocks goes through the 10,12 or 14 rounds depending on the key size.

The stages of each round of decryption are as follows :

- Add round key
- Inverse MixColumns
- ShiftRows
- Inverse SubByte

The decryption process is the encryption process done in reverse.

Key Expansion (Key Schedule)

In the **Advanced Encryption Standard (AES)**, Key Expansion (or Key Schedule) is the process of taking a single-bit master key and expanding it into a series of 128-bit **round keys**.

These round keys are used in the `AddRoundKey` step of each encryption and decryption round. A 128-bit key is expanded into **11 round keys** (1 for the initial transformation + 10 for the rounds), totaling 44 32-bit words.

Key Expansion Parameters

The number of rounds and words generated depends on the master key size:

AES Variant	Key Size (Bits)	Rounds	Round Keys Needed	Total Words
AES-128	128	10	11	44
AES-192	192	12	13	52
AES-256	256	14	15	60

The Core Algorithm (AES-128)

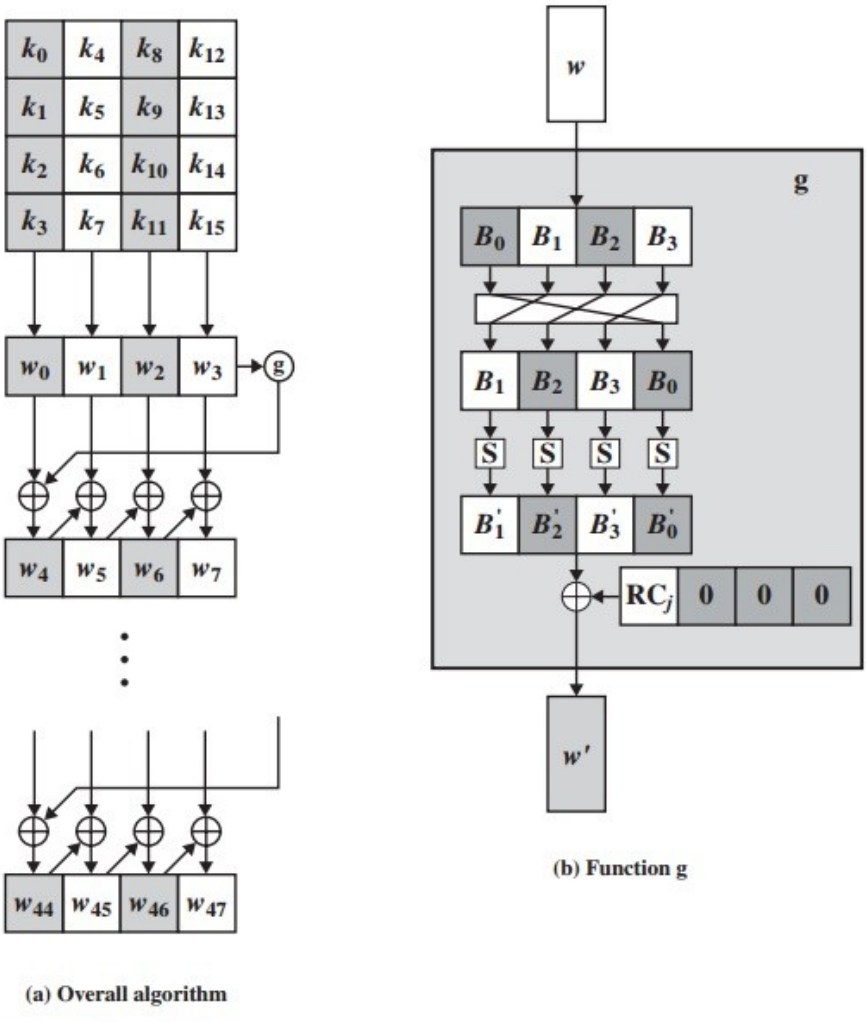


Figure 22: AES Key Generation Process

- The key is copied into the first four words of the expanded key.
- The remainder of the expanded key is filled in four words at a time.
- Each added word w[i] depends on the immediately preceding word, w[i - 1], and the word four positions back, w[i - 4].
- In three out of four cases, a simple XOR is used.
- For a word whose position in the w array is a multiple of 4, a more complex function(g) is used.
- The function g consists of the following subfunctions.
    - **RotWord:** Performs a 1-byte circular left shift. `[B0, B1, B2, B3]` becomes `[B1, B2, B3, B0]`.
    - **SubWord:** Substitutes each byte in the word using the **AES S-box** (the same one used in the main encryption rounds).
    - **Round Constant (Rcon):** XORs the first byte of the word with a round-dependent constant.

01 02 04 08 10 20 40 80 1B 36

PUBLIC-KEY CRYPTOSYSTEMS

Public-key cryptography represents a fundamental shift from traditional symmetric encryption. In symmetric systems, both sender and receiver must share a common secret key, which creates a major challenge in securely distributing that key.

Problem 1: Key Distribution

- In symmetric encryption:
    - Sender and receiver must share a **secret key**
    - Secure key exchange is difficult

Problem 2: Digital Signatures

- Symmetric systems **cannot provide non-repudiation**
- No way to prove sender identity securely

Public-key cryptography was proposed as a solution to both. Instead of sharing a secret key:

- Each user generates a **pair of keys**
- One key is **public**, the other is **private**

The public key is made openly available, while the private key is kept secret by the owner. These two keys are mathematically related, but the relationship is designed in such a way that it is computationally infeasible to derive the private key from the public key. This property forms the basis of security in such systems.

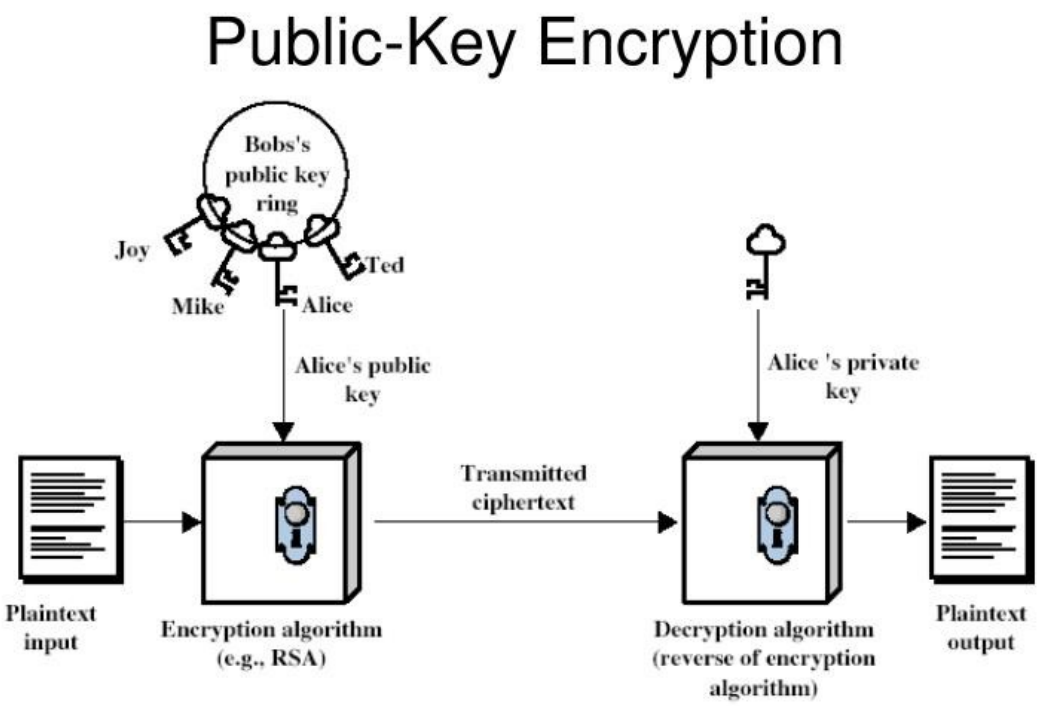


Figure 23: Encryption with Public Key

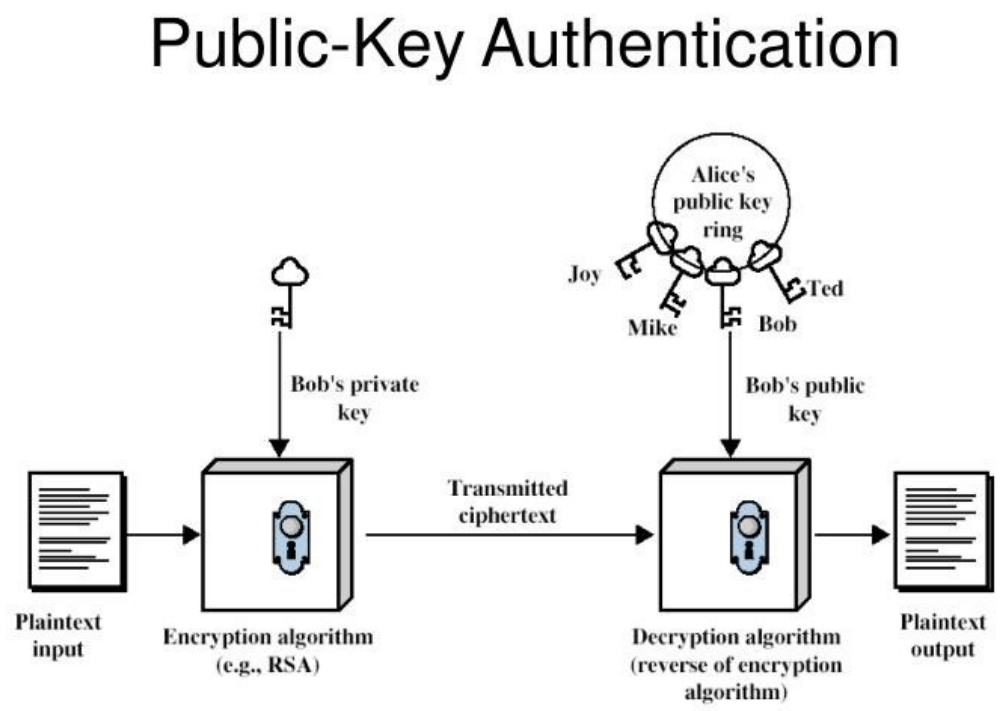


Figure 24: Encryption With Private Key

The working principle can be understood through two complementary operations. For ensuring confidentiality, a sender encrypts the message using the receiver's public key. Once encrypted, the

ciphertext can only be decrypted using the corresponding private key, which is known only to the receiver. This ensures that even if the communication is intercepted, the message remains secure. On the other hand, for authentication and digital signatures, the sender uses their own private key to encrypt (or sign) the message. The receiver can then use the sender's public key to verify the authenticity of the message. This dual capability is one of the most powerful features of public-key cryptography.

(A) Encryption for Confidentiality

- Sender uses **receiver's public key**

$C=E(KU_b,M)$

- Receiver uses **private key**

$M=D(KR_b,C)$

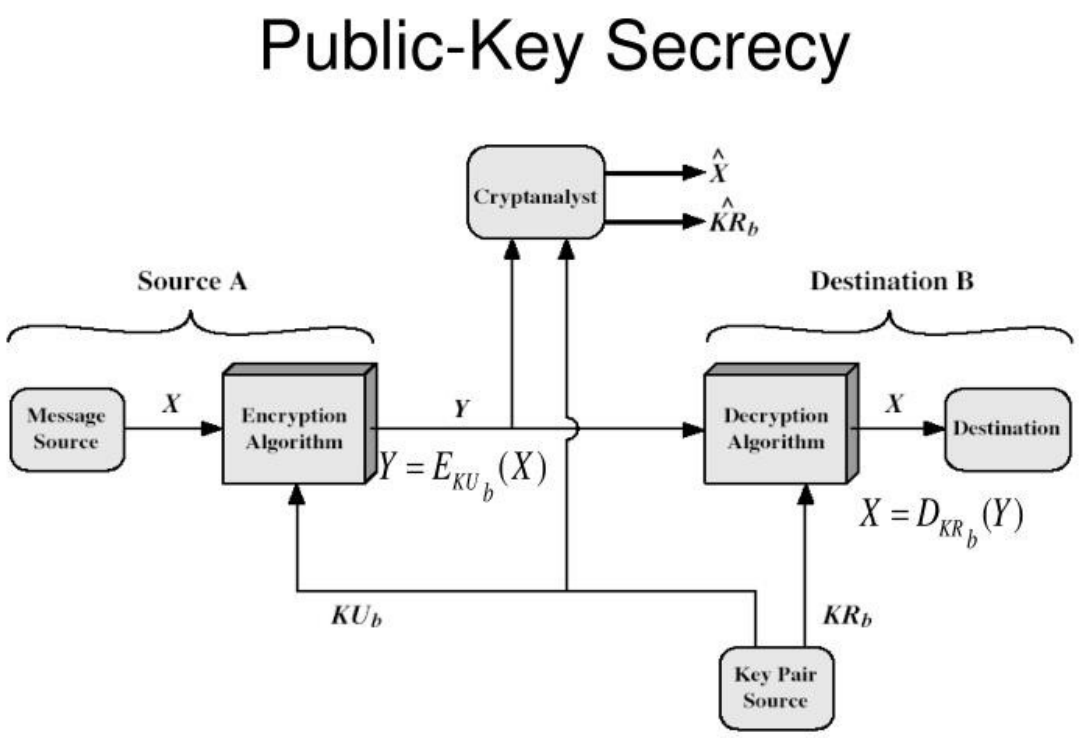


Figure 25: Confidentiality

(B) Authentication / Digital Signature

- Sender uses **own private key**

$S=E(KR_b,M)$

- Receiver verifies using **public key**

$M=D(KU_b,S)$

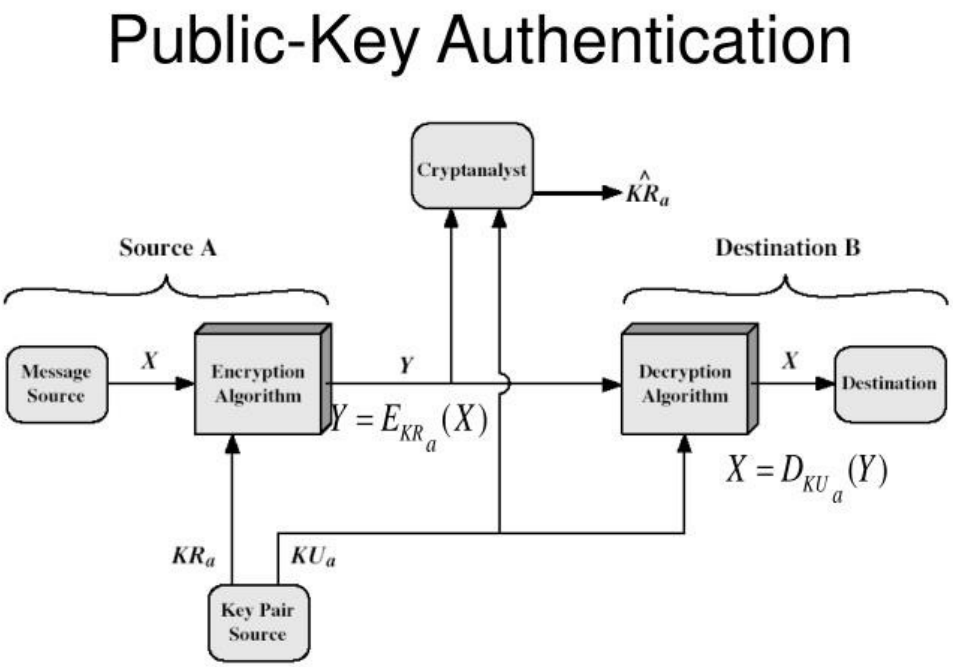


Figure 26: Authentication

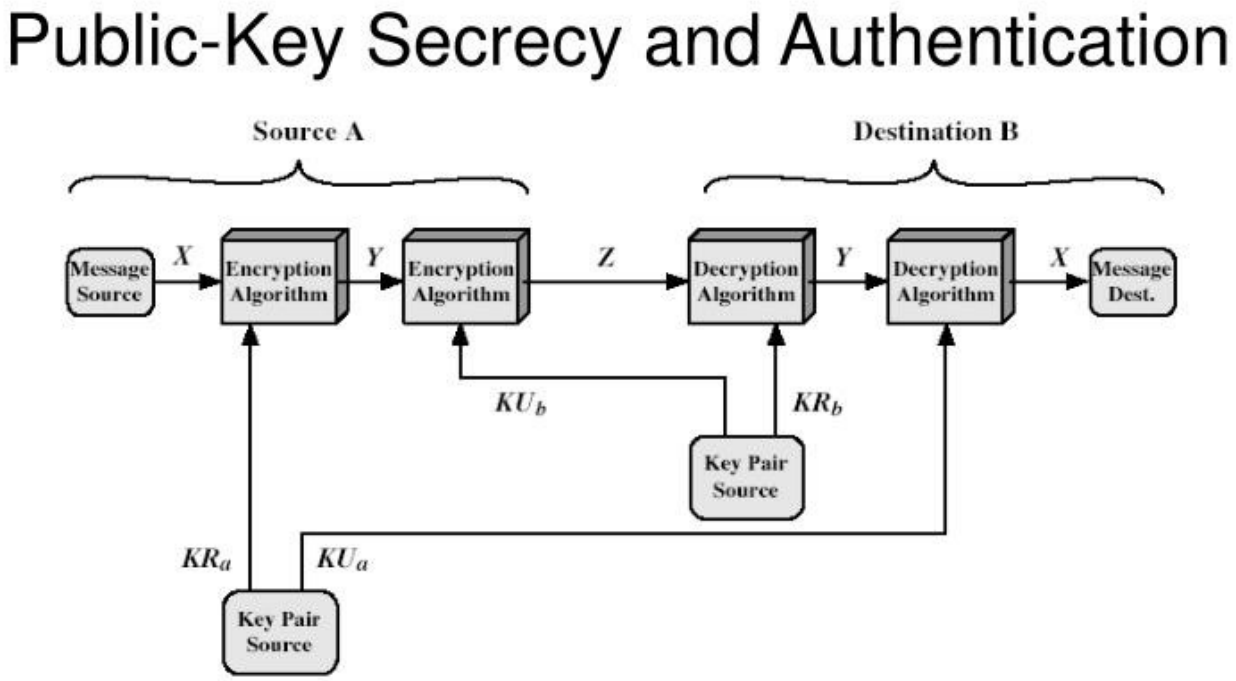


Figure 27: Confidentiality and Authentication

RSA Algorithm

The RSA algorithm is one of the most widely used public-key cryptosystems and is based on the mathematical difficulty of factoring large integers. It was developed by **Ron Rivest, Adi Shamir, and Leonard Adleman**, and it provides both encryption and digital signature capabilities. The fundamental idea behind RSA is that while it is easy to multiply two large prime numbers together, it is extremely difficult to factor their product back into the original primes. This asymmetry forms the basis of its security.

Key Generation

Step 1: Choose two large prime numbers

$$p \text{ and } q$$

Step 2: Calculate the product of primes

$$n = p \times q$$

Step 3: Compute Euler's Totient Function

$$\phi(n) = (p-1)(q-1)$$

Step 4: Choose public key exponent

$$e \text{ such that } 1 < e < \phi(n), \gcd(e, \phi(n)) = 1$$

Step 5: Compute private key exponent

$$d \text{ such that } d \times e \equiv 1 \pmod{\phi(n)}$$

Step 6: Form keys

- Public Key = (e,n)
- Private Key = (d,n)

Encryption Process

Step 1: Convert plaintext into integer

$$M < n$$

Step 2: Encrypt using public key

$$C = M^e \bmod n$$

Decryption Process

Step 1: Take ciphertext C

Step 2: Decrypt using private key

$$M = C^d \bmod n$$

Example:

Choose small primes (for understanding):

$$p = 3, \quad q = 11$$

$$n = p \times q = 3 \times 11 = 33$$

$$\phi(n) = (p-1)(q-1) = 2 \times 10 = 20$$

$$\gcd(7,20) = 1$$

$$e = 7$$

$$d \times e \equiv 1 \pmod{20}$$

$$7d \equiv 1 \pmod{20}$$

$$d = 3$$

Keys

- Public Key = $(e,n) = (7,33)$
- Private Key = $(d,n) = (3,33)$

Encryption

Given Message:

$$M = 2$$

Step:

$$C = M^e \bmod n$$

$$2^7 \bmod 33$$

$$128 \bmod 33 = 29$$

Ciphertext:

$$C = 29$$

Decryption

Step:

$$M = C^d \bmod n$$

$$29^3 \bmod 33$$

$$24389 \bmod 33 = 2$$

Decrypted Message:

$$M = 2$$

Diffie–Hellman Key Exchange Algorithm

The Diffie–Hellman algorithm is a fundamental public-key technique used for **secure key exchange over an insecure channel**. Unlike RSA, it is not used for direct encryption or decryption of messages; instead, it enables two parties to establish a **shared secret key** that can later be used with symmetric encryption algorithms. This method was introduced by **Whitfield Diffie and**

Martin Hellman and is based on the mathematical difficulty of solving the discrete logarithm problem.

The basic idea of the Diffie–Hellman algorithm is that two users can independently generate values and exchange certain information publicly, yet still arrive at the same secret key without ever transmitting that key directly. The security of the method lies in the fact that, although an attacker can see all exchanged values, it is computationally infeasible to determine the final shared secret.

Key Exchange Algorithm

Step 1: Choose a large prime number

$$q$$

Step 2: Choose a primitive root (generator) of q

$$\alpha$$

Step 3: User A selects a private key

$$X_A < q$$

Step 4: User B selects a private key

$$X_B < q$$

Step 5: User A computes public value

$$Y_A = \alpha^{XA} \bmod q$$

Step 6: User B computes public value

$$Y_B = \alpha^{XB} \bmod q$$

Step 7:

- A sends Y_A to B
- B sends Y_B to A

Step 8: User A computes

$$K=(Y_B)^{XA} \bmod q$$

Step 9: User B computes

$$K=(Y_A)^{XB} \bmod q$$

Example:

Choose public parameters:

$$q=23$$

Choose a primitive root α:

$$\alpha=5$$

Private Keys

User A chooses:

$$X_A = 6$$

User B chooses:

$$X_B = 15$$

Public Key Generation

For User A:

$$Y_A = \alpha^{XA} \bmod q$$

$$5^6 \bmod 23$$

$$Y_A = 8$$

For User B:

$$Y_b = \alpha^{Xb} \bmod q$$

$$5^{15} \bmod 23$$

$$Y_b = 19$$

Exchange Public Values

- A sends $Y_A = 8$ to B
- B sends $Y_B = 19$ to A

Shared Secret Key Computation

User A computes:

$$K=(Y_B)^{XA} \bmod q$$

$$19^6 \bmod 23$$

$$K = 2$$

User B computes:

$$K=(Y_A)^{XB} \bmod q$$

$$8^{15} \bmod 23$$

$$K = 2$$

Security Tools in Data Protection

Tools in the realm of data security and encryption are instrumental in fortifying the protection of sensitive information and ensuring the integrity of digital interactions.

- **Trusted Platform Module (TPM):** A TPM is a hardware-based security component embedded in modern computing devices. It is responsible for generating, storing, and

managing cryptographic keys within a secure environment. TPM ensures the integrity of the system during the boot process by verifying firmware and software components, thereby preventing unauthorized modifications. It also supports hardware-based authentication, making systems more secure against tampering and cyber threats.

- **Hardware Security Module (HSM)**: A HSM is a dedicated physical device designed to handle cryptographic operations such as encryption, decryption, and secure key management. It provides a highly secure environment where cryptographic keys are stored and used without ever leaving the device.
- **Key Management System (KMS):** A KMS is a software-based solution that centrally manages the lifecycle of cryptographic keys. It facilitates key generation, storage, distribution, rotation, and revocation, ensuring that keys are handled securely throughout their usage.
- **Secure Enclave** : A Secure Enclave is a hardware-isolated environment found within modern processors that is used to securely store sensitive data and execute cryptographic operations. It operates independently of the main operating system, thereby protecting sensitive information such as biometric data and encryption keys from software-based attacks.

Obfuscation Techniques

Obfuscation is a technique used to make code, data, or information more complex and difficult to understand. It is primarily used to prevent reverse engineering and protect intellectual property. By disguising the logic and structure of code, obfuscation adds an additional layer of security, making it harder for attackers to exploit vulnerabilities.

- **Steganography** is a method of hiding sensitive information within seemingly harmless digital content such as images or audio files. It works by subtly modifying the data in such a way that the hidden message remains undetectable to unauthorized users. This technique is commonly used for covert communication and digital watermarking.
- **Tokenization** is a process that replaces sensitive data with unique identifiers known as tokens. These tokens have no meaningful value outside the system, thereby reducing the risk of data exposure. Tokenization is widely used in payment systems where actual card details are replaced with tokens during transactions.
- **Data masking** involves replacing real data with fictitious but realistic values to protect sensitive information. This technique is particularly useful in testing and data analysis environments where real data is not required. It ensures that privacy is maintained while preserving the structure and usability of the dataset.

Hashing

Hashing is a cryptographic process that converts input data of any size into a fixed-length string known as a hash value. It is a one-way function, meaning that the original data cannot be reconstructed from the hash. Hash functions are designed to produce unique outputs for different inputs, minimizing the chances of collisions. Common hashing algorithms include MD5 and SHA-1. The two main reasons to use hashing are as follows:

- **Data integrity:** Hashing is widely used to ensure data integrity by comparing hash values before and after data transmission. If the values match, the data has not been altered.
- **Password security:** It is also extensively used in password security, where passwords are stored as hash values instead of plain text, making them more secure against unauthorized access.

Salting

Salting is a security technique used in conjunction with hashing to enhance password protection. It involves adding random data, known as a salt, to a password before hashing it. This makes each hashed password unique, even if multiple users have the same password. As a result, salting effectively prevents attacks such as rainbow table attacks and makes brute-force attempts significantly more difficult.

Digital Signatures

Digital signatures are cryptographic mechanisms that provide authentication, integrity, and non-repudiation in digital communications. They serve as the electronic equivalent of handwritten signatures but are far more secure. A digital signature is created by generating a hash of the document and encrypting it using the sender's private key. The receiver can then verify the signature using the sender's public key, ensuring that the document has not been altered and confirming the identity of the sender.

Key Stretching

Key stretching is a technique used to increase the security of passwords by making them harder to crack. It works by applying a hashing algorithm multiple times, thereby increasing the computational effort required to derive the original password. This slows down attackers attempting brute-force or dictionary attacks. Common key stretching techniques include PBKDF2 and Bcrypt.

- **Password-Based Key Derivation Function 2 (PBKDF2):** This widely used method iterates through a hash function multiple times, effectively slowing down the key derivation process.
- **Bcrypt:** Specifically designed to address password hashing, Bcrypt incorporates salt and multiple rounds of hashing to amplify the time required for each iteration

Blockchain

Blockchain is a decentralized digital ledger technology that records transactions across multiple computers in a secure and immutable manner. Each block in the chain contains data and a hash of the previous block, forming a continuous and tamper-resistant chain. Transactions are verified by network participants and added to the blockchain through consensus mechanisms such as proof of work. Due to its decentralized nature, blockchain ensures transparency, security, and trust in digital transactions.

Open Public Ledger

An open public ledger is a fundamental feature of blockchain technology that allows all participants in the network to access and verify transaction records. Unlike traditional centralized systems, the ledger is distributed across multiple nodes, ensuring that no single entity has complete control. Once a transaction is recorded, it becomes immutable and forms part of a chronological chain. Consensus

mechanisms are used to validate transactions, ensuring the integrity and accuracy of the ledger while promoting transparency and trust.

The benefits of the open public ledger are as follows:

- **Decentralization:** Unlike traditional centralized databases, the open public ledger is decentralized. Multiple copies of the ledger are distributed across nodes within a blockchain network.
- **Security:** Tampering with the ledger is immensely challenging, due to the decentralized and cryptographic nature of the system.
- **Transaction recording:** When a transaction occurs, it's broadcast to a network, where it is recorded. Network participants verify the transaction's validity, ensuring it adheres to the predefined rules of the blockchain.
- **Consensus mechanisms:** These mechanisms ensure that the network participants agree on the legitimacy of transactions before they are added to the ledger.
- **Immutable and chronological:** Once a transaction is validated and added to the ledger, it becomes a permanent part of the chain. Each block in the chain contains a reference to the previous block, creating a chronological sequence that's virtually tamper-proof.
- **Transparency:** The open nature of the ledger means that anyone can verify transactions independently. This transparency fosters trust and accountability within a network.

Digital Certificates

Digital certificates are essential for ensuring secure communication and protecting sensitive information in modern digital systems. They help verify identities and establish trust during online transactions, forming the backbone of secure data exchange over networks.

- **Certificate Authorities (CAs)** are trusted entities that issue and validate digital certificates using cryptographic techniques. They create a chain of trust through a root key, which signs certificates and links them back to a trusted root certificate. CAs can be online or offline, as well as public for internet use or private for internal organizational security.
- The **root** of trust is a key concept in Public Key Infrastructure (PKI), where a self-signed root certificate acts as the foundation of trust. Any certificate can be verified by tracing it back to this root certificate, ensuring authenticity and secure communication.
- Certificate validity is maintained using mechanisms such as **Certificate Revocation Lists (CRLs)** and the **Online Certificate Status Protocol (OCSP)**. CRLs provide lists of revoked certificates, while OCSP allows real-time validation by querying the CA directly, making it faster and more efficient.
- Self-signed certificates are generated by the same entity and are mainly used for internal purposes, while third-party certificates are issued by trusted CAs and are widely accepted for public use. A **Certificate Signing Request (CSR)** is generated when applying for a certificate and contains essential details like the public key and domain information.
- **Wildcard certificates** allow a single certificate to secure multiple subdomains under one domain, reducing cost and simplifying certificate management for organizations.